OS ORANGE 2

Name : Pulla Jagadeeshwar Reddy
SRN : PES1UG24CS347

Github link : https://github.com/Pulla-Jagadeeshwar-Reddy/PES1UG24CS347-PES-VCS

Phase 1:

1A

Output of ./test_objects showing all tests passing.

```
pes1ug24cs347@pes1ug24cs347:~/PES1UG24CS347-PES-VCS$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1B

find .pes/objects -type f showing the sharded directory structure.

```
pes1ug24cs347@pes1ug24cs347:~/PES1UG24CS347-PES-VCS$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
```

Phase 2 :

2A

Output of ./test_tree showing all tests passing

```
pes1ug24cs347@pes1ug24cs347:~/PES1UG24CS347-PES-VCS$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

2B

Pick a tree object from find .pes/objects -type f and run xxd .pes/objects/XX/YYY... | head -20 to show the raw binary format.

```
pes1ug24cs347@pes1ug24cs347:~/PES1UG24CS347-PES-VCS$ xxd .pes/objects/5d/378beca95c2cd6e43ea6beca3
663c5ea697ea6b5da7b026d2e1e595fc204e3 | head -20
00000000: 7472 6565 2039 3800 3130 3036 3634 2066  tree 98.100664 f
00000010: 696c 6531 2e74 7874 004c 466a 7c9a c7b2  ile1.txt.LFj|...
00000020: ae47 c01b 957e 02f8 b55e f81a 2af2 46c6  .G...~...^..*.F.
00000030: f165 423c 6309 0095 d231 3030 3636 3420  .eB<c....100664
00000040: 6669 6c65 322e 7478 7400 bb74 5365 9ced  file2.txt..tSe..
00000050: 8100 324e 1377 f9b4 388f 5740 236b b8be  ..2N.w..8.W@#k..
00000060: 1ffb 27d2 a749 4042 2963                 ..'..I@B)c
```

Phase 3:

3A

.Run ./pes init, ./pes add file1.txt file2.txt, ./pes status — show the output.

```
Initialized empty PES repository in .pes/
pes1ug24cs347@pes1ug24cs347:~/PES1UG24CS347-PES-VCS$ echo "hello" > file1.txt
pes1ug24cs347@pes1ug24cs347:~/PES1UG24CS347-PES-VCS$ echo "world" > file2.txt
pes1ug24cs347@pes1ug24cs347:~/PES1UG24CS347-PES-VCS$ ./pes add file1.txt file2.txt
pes1ug24cs347@pes1ug24cs347:~/PES1UG24CS347-PES-VCS$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  README.md
  untracked:  index.c
  untracked:  commit.h
  untracked:  .gitignore
  untracked:  tree.h
  untracked:  index.h
  untracked:  pes.c
  untracked:  tree.c
  untracked:  commit.c
  untracked:  hello.txt
  untracked:  Makefile
  untracked:  object.c
  untracked:  test_objects.c
  untracked:  pes.h
  untracked:  test_sequence.sh
  untracked:  test_tree.c
```

3B

cat .pes/index showing the text-format index with your entries.

```
pes1ug24cs347@pes1ug24cs347:~/PES1UG24CS347-PES-VCS$ cat .pes/index
100664 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776875734 6 file1.txt
100664 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776875739 6 file2.txt
```

Phase 4 :

4A

Output of ./pes log showing three commits with hashes, authors, timestamps, and messages.

```
pes1ug24cs347@pes1ug24cs347:~/PES1UG24CS347-PES-VCS$ ./pes init
Initialized empty PES repository in .pes/
pes1ug24cs347@pes1ug24cs347:~/PES1UG24CS347-PES-VCS$ echo "Hello" > hello.txt
pes1ug24cs347@pes1ug24cs347:~/PES1UG24CS347-PES-VCS$ ./pes add hello.txt
pes1ug24cs347@pes1ug24cs347:~/PES1UG24CS347-PES-VCS$ ./pes commit -m "Initial commit"
Committed: bae883716092... Initial commit
pes1ug24cs347@pes1ug24cs347:~/PES1UG24CS347-PES-VCS$ echo "World" >> hello.txt
pes1ug24cs347@pes1ug24cs347:~/PES1UG24CS347-PES-VCS$ ./pes add hello.txt
pes1ug24cs347@pes1ug24cs347:~/PES1UG24CS347-PES-VCS$ ./pes commit -m "Add world"
Committed: 100d9b2e6f80... Add world
pes1ug24cs347@pes1ug24cs347:~/PES1UG24CS347-PES-VCS$ ./pes log
commit 100d9b2e6f80f2f39ac3b23f3974c114994743b36b83b0fa7859209ef5fd78e4
Author: Pulla Jagadeeshwar Reddy <PES1UG24CS347>
Date:   1776875964

    Add world

commit bae8837160924e838803d50f14238eb69d48357756d041648d7395a50f67071d
Author: Pulla Jagadeeshwar Reddy <PES1UG24CS347>
Date:   1776875953

    Initial commit
```

4B

find .pes -type f | sort showing object store growth after three commits.

```
pes1ug24cs347@pes1ug24cs347:~/PES1UG24CS347-PES-VCS$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/10/0d9b2e6f80f2f39ac3b23f3974c114994743b36b83b0fa7859209ef5fd78e4
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/7c/83d6df2d741034233b181c4bfa7a6e64a49ec875cc62a850886d11af72429c
.pes/objects/ba/e8837160924e838803d50f14238eb69d48357756d041648d7395a50f67071d
.pes/objects/c1/5daab6ccf47288c923fa5384e07d0e7c63a4f443a1a77c77d51e67b257e822
.pes/refs/heads/main
```

4C

cat .pes/refs/heads/main and cat .pes/HEAD showing the reference chain.

```
pes1ug24cs347@pes1ug24cs347:~/PES1UG24CS347-PES-VCS$ cat .pes/refs/heads/main
100d9b2e6f80f2f39ac3b23f3974c114994743b36b83b0fa7859209ef5fd78e4
pes1ug24cs347@pes1ug24cs347:~/PES1UG24CS347-PES-VCS$ cat .pes/HEAD
ref: refs/heads/main
pes1ug24cs347@pes1ug24cs347:~/PES1UG24CS347-PES-VCS$
```

Phase 5 — Branching

Q5.1 — pes checkout <branch>

In PES-VCS, a branch is basically just a file inside .pes/refs/heads/ that stores a commit hash. So when implementing pes checkout <branch>, the first step is to update the .pes/HEAD file so that it points to the selected branch. After that, the system reads the commit of that branch, gets its tree, and rebuilds the working directory to match that snapshot. This means writing all the files from that commit and removing any files that don't belong to it.

One tricky part here is handling uncommitted changes. If the user has modified files and we overwrite them, their work would be lost, so the checkout should stop in such cases. Also, different branches may have different directory structures, so we need to handle creating and deleting folders properly. Finally, the index also needs to be updated so that everything looks clean after switching branches.

Q5.2 — Detecting a dirty working directory

To check if the working directory is dirty during checkout, we first compare the current commit with the target commit and identify which files are different. For those files, we check their metadata in the working directory and compare it with what's stored in the index. If the file's size or modification time doesn't match, it means the file has been changed after staging.

In that case, we treat it as a conflict and stop the checkout to avoid losing changes. This method is efficient because it doesn't require reading or hashing file contents — it just uses metadata like size and time.

Q5.3 — Detached HEAD

A detached HEAD happens when .pes/HEAD directly stores a commit hash instead of pointing to a branch. In this state, if we make new commits, they are created normally and stored in the object store, but no branch points to them.

Because of that, once we switch back to a branch, those commits are no longer referenced and can eventually be lost. However, if we remember the commit hash, we can still recover them by creating a new branch and pointing it to that hash manually.

Phase 6 — Garbage Collection

Q6.1 — Finding unreachable objects

To remove unused objects, we can use a mark-and-sweep approach. First, in the mark phase, we start from all branch heads and follow their commits, trees, and blobs, marking everything that is reachable. We can store these hashes in something like a hash set for quick lookup.

Then in the sweep phase, we go through all objects in .pes/objects/ and delete the ones that are not in the reachable set. This way, only objects that are no longer used by any branch get removed, which helps save space.

Q6.2 — Race condition between GC and commits

A race condition can happen if garbage collection runs at the same time as a commit. For example, a new object might be created during a commit but not yet linked to any branch. If GC runs at that moment, it may think the object is unused and delete it, which can break the repository.

To avoid this, systems like Git don’t delete recently created objects and also use locking to prevent such conflicts. This ensures that objects created during a commit are safe and won’t be accidentally removed.